



Please solve all these questions given in the attached PDF with good understandable explanations

I'll solve all the Advanced Dynamic Programming problems from your PDF with clear, understandable explanations.^[1]

Problem 1: Royal Chef's Platter Arrangement

Problem Understanding

You have a sequence of dishes (savory=1, sweet=2) that must be distributed across K platters while maintaining order. The goal is to minimize the "interaction score" - the sum of (savory count $\times$ sweet count) for each platter.^[1]

Dynamic Programming Approach

State Definition: Let $dp[i][k]$ = minimum interaction score to place the first i dishes onto exactly k platters.^[1]

Base Cases:

- $dp = 0$ (no dishes, no platters)
- $dp[i] = \infty$ for $i > 0$ (dishes but no platters - impossible)
- $dp[k] = \infty$ for $k > 0$ (platters but no dishes - invalid)^[1]

Recurrence: For each position i and platter count k , try all possible ways to form the last platter:
 $dp[i][k] = \min_{j < i} \{dp[j][k-1] + \text{score}(j+1, i)\}$

where $\text{score}(j+1, i)$ is the interaction score for dishes from position $j+1$ to i (count of 1's $\times$ count of 2's in that range).^[1]

Algorithm:

1. Precompute prefix counts of savory and sweet dishes
2. Initialize dp array with infinity, $dp[0][0] = 0$
3. For i from 1 to n :
 - For k from 1 to $\min(i, K)$:
 - For j from $k-1$ to $i-1$:
 - savory = count of 1's in dishes[j+1...i]
 - sweet = count of 2's in dishes[j+1...i]
 - score = savory $\times$ sweet

```

        dp[i][k] = min(dp[i][k], dp[j][k-1] + score)
4. Return dp[n][K] if dp[n][K] < infinity, else -1

```

Complexity:

- Time: $O(n^2 \cdot K)$ - three nested loops
- Space: $O(n \cdot K)$ for the DP table^[1]

Example: For dishes = {1,2,1,1,2}, K = 2:

- Split {1,2} | {1,1,2}: score = 1×1 + 2×1 = 3
- The algorithm systematically tries all valid splits and picks the minimum^[1]

Problem 2: Traveler's Minimum Energy

Problem Understanding

A traveler moves through an M×N grid from top-left to bottom-right, moving only right or down. Each cell changes their energy (negative drains, positive restores). Find the minimum starting energy needed so energy never drops to zero or below.^[1]

Key Insight

We need to work **backwards** from the destination! At each cell, calculate the minimum energy required to reach the destination from that cell.^[1]

Dynamic Programming Approach

State Definition: Let $\text{minEnergy}[i][j]$ = minimum energy required **at the start** of cell (i,j) to successfully reach the destination.^[1]

Base Case:

$\text{minEnergy}[M-1][N-1] = \max(1, 1 - \text{grid}[M-1][N-1])$

This ensures we have at least 1 energy after visiting the last cell.^[1]

Recurrence (working backwards):

For each cell (i,j), we can go to (i+1,j) or (i,j+1). The minimum energy needed at (i,j) is:

$\text{minEnergy}[i][j] = \max(1, \min(\text{minEnergy}[i+1][j], \text{minEnergy}[i][j+1]) - \text{grid}[i][j])$

Explanation:

- We need enough energy to survive the next cell minus what we gain/lose in current cell
- We always need at least 1 energy to be alive^[1]

Algorithm:

```

1. Create minEnergy array of size M × N
2. Set minEnergy[M-1][N-1] = max(1, 1 - grid[M-1][N-1])
3. Fill last column (going up):
   For i from M-2 down to 0:

```

```

        minEnergy[i][N-1] = max(1, minEnergy[i+1][N-1] - grid[i][N-1])
4. Fill last row (going left):
    For j from N-2 down to 0:
        minEnergy[M-1][j] = max(1, minEnergy[M-1][j+1] - grid[M-1][j])
5. Fill rest of table (bottom-up, right-to-left):
    For i from M-2 down to 0:
        For j from N-2 down to 0:
            right = minEnergy[i][j+1]
            down = minEnergy[i+1][j]
            minEnergy[i][j] = max(1, min(right, down) - grid[i][j])
6. Return minEnergy[0][0]

```

Complexity:

- Time: $O(M \times N)$
- Space: $O(M \times N)$ (can be optimized to $O(N)$ using rolling array)^[1]

Example Walkthrough: For the given grid:

```

-2  -4  15
-1  -5  -4
-4  20  -1

```

Working backwards:

- At (2,2): need $\max(1, 1 - (-1)) = 2$
- At (2,1): need $\max(1, 2 - 20) = 1$
- Continue until reaching (0,0) = 7^[1]

Problem 3: Extended Edit Distance

Problem 3(a): Algorithm Design

State Definition: Let $dp[i][j]$ = minimum cost to transform $x[1..i]$ into $y[1..j]$.^[1]

Operations and Transitions:

1. **Copy** (cost c_{copy}): If $x[i] = y[j]$, $dp[i][j] = dp[i-1][j-1] + c_{\text{copy}}$
2. **Replace** (cost c_{replace}): $dp[i][j] = dp[i-1][j-1] + c_{\text{replace}}$
3. **Delete** (cost c_{delete}): $dp[i][j] = dp[i-1][j] + c_{\text{delete}}$
4. **Insert** (cost c_{insert}): $dp[i][j] = dp[i][j-1] + c_{\text{insert}}$
5. **Twiddle** (cost c_{twiddle}): If $x[i-1] = y[j]$ and $x[i] = y[j-1]$,
 $dp[i][j] = dp[i-2][j-2] + c_{\text{twiddle}}$
6. **Kill** (cost c_{kill}): $dp[m][j] = c_{\text{kill}}$ if we're at the end of x and want to ignore remaining characters^[1]

Recurrence:

$$dp[i][j] = \min \begin{cases} dp[i-1][j-1] + \text{match_cost}(x[i], y[j]) & (\text{copy/replace}) \\ dp[i-1][j] + c_{\text{delete}} & (\text{delete}) \\ dp[i][j-1] + c_{\text{insert}} & (\text{insert}) \\ dp[i-2][j-2] + c_{\text{twiddle}} & (\text{twiddle, if valid}) \\ c_{\text{kill}} & (\text{kill, if } i = m) \end{cases}$$

where $\text{match_cost}(x[i], y[j]) = c_{\text{copy}}$ if $x[i] = y[j]$, else c_{replace} .^[1]

Algorithm:

1. Initialize $dp[0][0] = 0$
2. Initialize first row: $dp[0][j] = j \times c_{\text{insert}}$
3. Initialize first column: $dp[i][0] = i \times c_{\text{delete}}$ (or c_{kill} if allowed)
4. For i from 1 to m :
 - For j from 1 to n :
 - Calculate minimum from all applicable operations
 - Store which operation was used in $\text{parent}[i][j]$
5. Backtrack from $dp[m][n]$ using parent array to print operations
6. Return $dp[m][n]$

Complexity:

- Time: $O(m \times n)$
- Space: $O(m \times n)$ for DP table and parent pointers^[1]

Problem 3(b): Sequence Alignment

Sequence alignment in bioinformatics aligns two DNA/protein sequences to find similarities.^[1]

Casting as Edit Distance:

- **Match** (characters are same): Use **Copy** operation with cost 0 or negative (reward)
- **Mismatch** (characters differ): Use **Replace** with positive cost (penalty)
- **Gap in first sequence**: Use **Insert** with gap penalty
- **Gap in second sequence**: Use **Delete** with gap penalty^[1]

Operations used: Copy, Replace, Insert, Delete (no Twiddle or Kill needed).^[1]

Alignment score: Traditional alignment maximizes similarity score, which is equivalent to minimizing edit distance with appropriately negated costs.^[1]

Problem 4: Increasing Back-and-Forth Subsequence

Problem Understanding

Find the longest subsequence where values strictly increase, and indices alternate direction: left on odd steps, right on even steps.^[1]

Dynamic Programming Approach

State Definition:

- $dpLeft[i]$ = length of longest back-and-forth subsequence ending at position i where the **next move should be left** (current step is even)
- $dpRight[i]$ = length of longest back-and-forth subsequence ending at position i where the **next move should be right** (current step is odd)^[1]

Base Case:

- $dpLeft[i] = 1$ for all i (single element)
- $dpRight[i] = 1$ for all i ^[1]

Recurrence:

For each position i :

- $dpRight[i] = \max_{j>i, A[j]>A[i]} (dpLeft[j] + 1)$ (move right to position j where next move is left)
- $dpLeft[i] = \max_{j<i, A[j]>A[i]} (dpRight[j] + 1)$ (move left to position j where next move is right)^[1]

Algorithm:

```
1. Initialize  $dpLeft[i] = 1, dpRight[i] = 1$  for all  $i$ 
2. For  $i$  from 0 to  $n-1$ :
    For  $j$  from  $i+1$  to  $n-1$ :
        if  $A[j] > A[i]$ :
             $dpRight[i] = \max(dpRight[i], dpLeft[j] + 1)$ 
    For  $j$  from 0 to  $i-1$ :
        if  $A[j] > A[i]$ :
             $dpLeft[i] = \max(dpLeft[i], dpRight[j] + 1)$ 
3. Return  $\max(\max(dpLeft), \max(dpRight))$ 
```

Optimization: Process in a smart order to avoid redundant computations. Can be done more efficiently with segment trees.^[1]

Complexity:

- Time: $O(n^2)$
- Space: $O(n)$ ^[1]

Example: Array = ^[2] ^[1]

- Start at index 1 (value 1), move right to index 2 (value 4), move left to index 0 (value 3 - invalid)
- Start at index 3 (value 1), move right to index 4 (value 5), move left to index 2 (value 4 - invalid)
- Try different starting positions to find longest valid subsequence^[1]

Problem 5: Text Justification with Minimum Slop

Problem Understanding

Break a paragraph into lines of length L , minimizing the sum of squared extra spaces (excluding the last line).^[1]

Dynamic Programming Approach

State Definition: Let $dp[i]$ = minimum total slop for words 1 through i .^[1]

Base Case: $dp = 0$ (no words, no slop).^[1]

Recurrence: For each word i , try all possible starting positions j for the line containing word i :

$$dp[i] = \min_{j < i} \begin{cases} dp[j] + \text{slop}(j+1, i) & \text{if words } j+1..i \text{ fit on a line and } i < n \\ dp[j] & \text{if } i = n \text{ (last line, no slop counted)} \end{cases}$$

Validity Check: Words $j+1$ through i fit on a line if:

$$\sum_{k=j+1}^i \ell_k + (i - j - 1) \leq L$$

Slop Calculation: If words $j+1$ through i form a line (not the last):

$$\text{slop}(j+1, i) = \left(L - (i - j - 1) - \sum_{k=j+1}^i \ell_k \right)^2$$

Algorithm:

```
1. Precompute prefix sums: prefixLen[i] = sum of lengths of first i words
2. Initialize dp[0] = 0, dp[i] = infinity for i > 0
3. For i from 1 to n:
    For j from 0 to i-1:
        totalWordLen = prefixLen[i] - prefixLen[j]
        numSpaces = i - j - 1
        if totalWordLen + numSpaces <= L:
            extraSpace = L - totalWordLen - numSpaces
            if i == n: // Last line, no slop
                dp[i] = min(dp[i], dp[j])
            else:
                dp[i] = min(dp[i], dp[j] + extraSpace2)
4. Return dp[n]
```

Optimization: Use prefix sums to compute word length sums in $O(1)$ time.^[1]

Complexity:

- Time: $O(n^2)$ - for each word, try all possible line starts
- Space: $O(n)$ for DP array and prefix sums^[1]

Example: Words with lengths $[4, 3, 1]$, $L = 10$ ^[1]

- Line 1: words 1-2 (4+3+1 space = 8, extra = 2, slop = 4)

- Line 2: words 3-4 ($2+5+1$ space = 8, extra = 2, slop = 4)
- Line 3: word 5 (6, last line, no slop)
- Total slop = 8^[1]

Backtracking for Solution: Maintain a parent array to record which j was chosen for each i , then backtrack to reconstruct the optimal line breaks.^[1]

All solutions use dynamic programming principles: optimal substructure and overlapping subproblems. The key is identifying the right state representation and recurrence relation for each problem.^[1]

**

1. AAD_QB_1_4-Advanced-DP.pdf
2. AAD_QB_1_5-All-Topics.pdf